Merge Sort

1. Source Code (C)

```c
#include <stdio.h>
#include <stdlib.h>

void merge(int arr[], int l, int m, int r)
{
    int i, j, k;
    int n1 = m - l + 1;
    int n2 = r - m;

    int L[n1], R[n2];

    for (i = 0; i < n1; i++)
        L[i] = arr[l + i];

    for (j = 0; j < n2; j++)
        R[j] = arr[m + 1 + j];

    i = 0;
    j = 0;
    k = l;

    while (i < n1 && j < n2)
    {
        if (L[i] <= R[j])
        {
            arr[k] = L[i];
            i++;
        }
        else
        {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    while (i < n1)
    {
        arr[k] = L[i];
        i++;
        k++;
    }

    while (j < n2)
    {
        arr[k] = R[j];
        j++;
        k++;
    }
}

void mergeSort(int arr[], int l, int r)
{
    if (l < r)
    {
        int m = l + (r - l) / 2;

        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);

        merge(arr, l, m, r);
```

```
    }
}

int main()
{
    int arr[] = {38, 27, 43, 10};
    int arr_size = sizeof(arr) / sizeof(arr[0]);
    int i;

    printf("before sorting \n");

    for (i = 0; i < arr_size; i++)
        printf("%d ", arr[i]);

    printf("\n");

    mergeSort(arr, 0, arr_size - 1);

    printf("after sorting \n");

    for (i = 0; i < arr_size; i++)
        printf("%d ", arr[i]);

    printf("\n");

    return 0;
}
```

2. Program Output

```
before sorting
38 27 43 10
after sorting
10 27 38 43
```